

Anomaly Detection - Project Report

1. Introduction

Network intrusion detection remains a core security task because modern networks generate heterogeneous, high-volume traffic where malicious activity can be sparse, rapidly evolving, and difficult to separate from benign behaviour. In this context, intrusion detection systems (IDS) must balance two competing operational goals: detecting attacks with high sensitivity while limiting false alarms that can overwhelm analysts or automated response pipelines. Data-driven approaches—especially supervised classification and anomaly detection—are widely used because they can exploit statistical patterns in network-flow features that are not easily captured by hand-crafted rules.

This report investigates intrusion detection using the **UNSW-NB15** dataset, a benchmark collection of network traffic flows labelled as normal or attack and accompanied by rich feature engineering. The dataset was introduced to mitigate well-known limitations of older IDS benchmarks that may no longer reflect contemporary traffic characteristics and “low-footprint” attack behaviours. In the original dataset construction, a synthetic testbed environment was established and the **IXIA PerfectStorm** tool was used to generate a hybrid of modern benign activities and contemporary malicious behaviours, while raw traffic was captured using **tcpdump** and later processed into flow-based records. Feature engineering combined established network monitoring tooling—most notably **Argus** and **Bro-IDS**—with additional algorithms that compute deeper connection statistics, resulting in a feature set designed to characterise both packet-level and flow-level dynamics relevant for intrusion detection.

A key strength of UNSW-NB15 is that it supports both binary and multi-category security modelling. At the label level, the dataset provides a binary target (normal vs. attack) and an attack-category field, where attacks are organised into **nine families: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms**. The official dataset documentation likewise highlights this nine-family structure and describes the dataset as a hybrid of real modern normal activities and synthetic contemporary attack behaviours generated in a controlled cyber range setting. Taken together, these properties make UNSW-NB15 suitable for comparing supervised classifiers (trained on labelled attack examples) with anomaly detectors (trained only on benign traffic to detect deviations).

In this notebook-based workflow, the dataset is loaded from predefined splits: the **training set contains 175,341 records** and the **testing set contains 82,332 records**, each provided as structured CSV files. A separate feature metadata file is also available, supporting a consistent interpretation of feature roles, names, and types across the pipeline. The presence of a fixed train/test split is particularly important for methodological validity because it enables a clear separation between model development and out-of-sample evaluation, reducing the risk of optimistic conclusions driven by inadvertent leakage or post-hoc thresholding on test data.

Beyond its breadth, UNSW-NB15 is also known to contain structural challenges that influence how IDS models should be trained and interpreted. Prior analysis through visualization identifies **class imbalance** and **class overlap** as two major issues that can hinder attack detection and identification if not explicitly addressed during preprocessing and modelling. In practice, imbalance can bias learners toward dominant classes and inflate accuracy without ensuring adequate attack coverage, while overlap implies that certain attack families may not be cleanly separable from benign traffic in the feature space, limiting achievable performance regardless of model complexity. For these reasons, careful preprocessing, appropriate metrics (including threshold-sensitive measures), and explicit tuning procedures are necessary to produce reliable and operationally meaningful results.

The central objective of this work is to build a structured IDS modelling pipeline that enables a principled comparison between **supervised intrusion classification** and **unsupervised/anomaly detection** under consistent preprocessing and evaluation conditions. The supervised branch learns a direct mapping from preprocessed network-flow features to the binary target label, making it suitable when representative labelled attacks are available. In parallel, the unsupervised/anomaly branch is trained using normal-only traffic to learn a baseline model of benign behaviour; attacks are then detected as deviations from this baseline using anomaly scoring and reconstruction error mechanisms. This dual formulation reflects realistic deployment constraints, where labels may be incomplete and attack patterns may drift over time, and it directly aligns with the assignment's requirement to compare classical anomaly detection, supervised classification, and a deep learning autoencoder.

To ensure clarity and reproducibility, the report follows a fixed structure aligned explicitly with the notebook sections. The **Introduction** corresponds to notebook section **1)** and establishes the IDS motivation, dataset context, objectives, and report roadmap. **Data Pre-processing** corresponds to notebook sections **2)–7)** and documents data loading and integrity checks, label structure and imbalance analysis, feature typing, exploratory summaries (including distributional diagnostics and correlation structure), dataset branching into supervised vs. normal-only regimes, and the construction of leakage-safe preprocessing pipelines. **Modelling** corresponds to notebook sections **9), 10), 12), and 14)** and details the supervised baselines (including imbalance-aware variants), classical anomaly detectors, and the deep autoencoder design, focusing on modelling rationale and numeric setup. **Evaluation** draws on the same modelling sections **(9), 10), 12), and 14)** but concentrates specifically on performance artefacts and trade-offs (classification metrics, confusion matrices, ROC/PR curves, and score/error distributions). **Model Tuning** corresponds to notebook sections **11) and 13)** and explains hyperparameter search and threshold calibration procedures and how they affect measurable performance. **Model Selection** corresponds to notebook section **15)** and justifies the final model choice by balancing empirical results with operational considerations (e.g., the cost of false negatives vs. false positives in security contexts). The report then concludes by summarising the main findings and highlighting practical improvements and future work directions for IDS modelling on UNSW-NB15.

To support transparency and reproducibility, a dedicated **Resources** chapter is included at the end of this report, containing the complete list of all books, scientific articles, and official web links consulted throughout the study.

2. Data Pre-processing

The data pre-processing stage establishes a reliable foundation for modelling by validating dataset integrity, analysing target structure and imbalance, characterising feature types and distributions, and constructing leakage-safe preprocessing pipelines for both supervised and anomaly-detection workflows. All checks and transformations reported in this section correspond directly to notebook steps **2)–7)** and are designed to ensure that any downstream modelling differences reflect algorithmic behaviour rather than artefacts such as duplicated rows, missing values, inconsistent encoding, or train–test leakage.

2.1. Data integrity checks: duplicates and missing values

The first step in pre-processing was a basic integrity audit of the training and test splits. Duplicate checks show that the dataset contains **0 duplicated rows in the training set** and **0 duplicated rows in the test set**. This is important in intrusion detection settings because duplicated samples can artificially inflate apparent learnability (e.g., by allowing models to memorise repeated flows) and can bias evaluation if duplicates occur across splits.

Missing-value analysis further indicates that the dataset is complete for the features examined. The notebook prints the “Top NaN columns” for both train and test, and all reported counts are **0**. Because the list is sorted in descending order, having zero missing values at the top implies that missingness is not a driver of downstream results and that no imputation strategy is required. As a consequence, the modelling pipeline can focus on encoding and scaling rather than handling null patterns, and comparisons between supervised and anomaly-based methods remain uncontaminated by imputation design choices.

2.2. Targets and class balance: label and attack categories

After confirming data integrity, the pre-processing stage explicitly characterises the target variable and its imbalance structure. The binary target `label` is defined as **0 = normal** and **1 = attack**. In the **training split**, the class distribution is **119,341 attack samples** and **56,000 normal samples**, corresponding to proportions of approximately **68.06% attacks** and **31.94% normals**. This imbalance is non-trivial and is a central practical feature of intrusion detection: many models can achieve deceptively high overall accuracy simply by favouring the majority class. For this reason, the report later relies on threshold-sensitive diagnostics and precision–recall analysis in the Evaluation chapter, but the imbalance itself is established here as a core dataset property that pre-processing must account for.

In addition to the binary label, the dataset provides an `attack_cat` field that differentiates attack families. The notebook reports the top categories for both splits. In the **training set**, the largest categories are **Normal (56,000)**, **Generic (40,000)**, **Exploits (33,393)**, **Fuzzers (18,184)**, **DoS (12,264)**, and **Reconnaissance (10,491)**, followed by smaller classes such as **Analysis (2,000)**, **Backdoor (1,746)**, **Shellcode (1,133)**, and **Worms (130)**. A similar long-tail structure is present in the **test set**, with **Normal (37,000)**, **Generic (18,871)**, **Exploits (11,132)**, **Fuzzers (6,062)**, **DoS (4,089)**, and **Reconnaissance (3,496)**, followed by **Analysis (677)**, **Backdoor (583)**, **Shellcode (378)**, and **Worms (44)**. This distribution

highlights two important pre-processing implications: (1) while the binary label is used for modelling in this assignment, the underlying attack space is heterogeneous (different mechanisms, different feature signatures), and (2) the extreme rarity of some categories (e.g., Worms) suggests that attack diversity and long-tail patterns can make generalisation difficult, particularly for anomaly detectors trained on “normal-only” data.

The notebook also produces two figures in this section: a label distribution plot and an attack-category distribution plot. These visualisations reinforce the numeric counts by showing the dominance of the attack class in training and the steep drop-off from the most common categories to rare classes. In reporting terms, these figures justify later workflow decisions such as using imbalance-aware strategies (e.g., SMOTE for supervised training) and emphasising precision–recall diagnostics in the Evaluation chapter.

2.3. Feature typing: categorical vs numerical predictors

To build a robust preprocessing pipeline, the notebook explicitly separates predictors into categorical and numerical groups. The workflow identifies three categorical variables: **proto**, **service**, and **state**, while confirming that the modelling feature set contains **42 total feature columns**, of which **39 are numerical**. This split is crucial because categorical fields require encoding (rather than scaling), and improper handling (e.g., treating categories as numeric codes) can introduce artificial ordinality and degrade both supervised and anomaly models.

Cardinality analysis shows meaningful complexity in network metadata. The protocol feature **proto** has **133 unique values** in the training set, indicating that protocol diversity is substantial and that one-hot encoding will expand dimensionality. The notebook prints the most frequent **proto** values, with **tcp (79,946)** and **udp (63,283)** dominating, followed by less common values such as **unas (12,084)**, **arp (2,859)**, **ospf (2,595)**, and smaller counts for others. The **service** feature has **13 unique values**, with the placeholder “-” (**94,168**) and common services such as **dns (47,294)** and **http (18,724)** appearing among the most frequent. Finally, **state** has **9 unique values**, with **INT (82,275)** and **FIN (77,825)** accounting for the majority of observations, followed by smaller classes such as **CON (13,152)** and **REQ (1,991)**, and very rare states (e.g., single-digit counts). From a preprocessing perspective, these distributions matter for two reasons. First, high-cardinality fields like **proto** can contribute many sparse indicator columns after encoding, which can affect the behaviour of linear models and distance-based anomaly detectors. Second, strong dominance of a few categories suggests that rare categories may behave like outliers in feature space, potentially influencing anomaly scores if not handled carefully.

2.4. Numerical feature exploration: descriptive statistics and correlation structure

The notebook performs a compact exploratory analysis of numerical variables using descriptive statistics and a correlation heatmap over a subset of key numeric features. The descriptive table reveals that many network-flow features are **highly skewed** and **heavy-tailed**, which is typical in traffic data where most flows are short and small, but a

minority can be extremely large. For example, the duration feature `dur` has a mean around **1.36** while its median is close to **0.0016**, indicating that most flows are very short but a small number of long flows inflate the mean; the maximum duration reaches approximately **60**. Packet and byte counters exhibit similar behaviour: `spkts` and `dpkts` have medians around **2**, while their maxima reach **9,616** and **10,974**, respectively, and byte-based features such as `sbytes` and `dbytes` have maxima on the order of 10^7 , far above their typical quartile values. This numeric profile supports two preprocessing decisions that are essential for stable modelling: (1) applying feature scaling for models sensitive to magnitude (e.g., SVM, kNN-based anomaly scoring, and neural networks), and (2) avoiding assumptions of normality for raw feature distributions, since many variables are far from Gaussian.

The correlation heatmap figure further clarifies relationships among network metrics and indicates where redundancy or structural dependency may exist. The heatmap shows clear correlated blocks among traffic volume and packet-count features (e.g., `spkts`, `dpkts`, `sbytes`, `dbytes`), as well as structured relationships among TCP-related variables and window/handshake indicators (`swin`, `dwin`, `stcpb`, `dtcpb`, `tcprrt`, `synack`, `ackdat`). Such correlations are meaningful for downstream modelling: tree ensembles can exploit correlated predictors without strict assumptions, while distance-based anomaly detectors and reconstruction models can be influenced by correlated clusters that dominate variance. The presence of structured correlations also provides intuition for why reconstruction-based approaches (PCA and autoencoders) can be viable: if normal traffic lies on a constrained manifold shaped by protocol mechanics and traffic dynamics, reconstruction error can capture deviations from that manifold.

2.5. Dataset branching: supervised vs normal-only anomaly workflows

Because the report compares supervised classification with anomaly detection, the pre-processing stage explicitly defines two dataset branches that reflect different learning assumptions. The **supervised branch** uses the full labelled training data and retains the binary labels for standard classification training. In numeric terms, this produces a supervised training matrix of $(175,341 \times 42)$ with a label vector of **175,341** entries, and a mixed test matrix of $(82,332 \times 42)$ with its corresponding test labels. In contrast, the **normal-only branch** isolates benign traffic to train anomaly detectors that model “normality” without exposure to attacks. This branch contains **56,000 normal training samples** with the same **42 features**. Establishing these branches at pre-processing time is essential for methodological validity: anomaly models trained on normal-only data should not be influenced by attack examples, and comparisons later in the report must account for the fact that supervised and unsupervised methods are trained on fundamentally different data regimes.

2.6. Leakage-safe preprocessing: ColumnTransformer pipelines and resulting dimensionality

The final step in the pre-processing chapter is the construction of reproducible preprocessing pipelines using a **ColumnTransformer** approach that applies transformations by feature

type. Categorical variables (**proto**, **service**, **state**) are one-hot encoded, while numerical variables are scaled to ensure comparable magnitudes for distance-based, margin-based, and neural models. Importantly, the preprocessing steps are fitted on training data only and then applied to the test split, preserving leakage safety. This is particularly important given the high cardinality of **proto** and the strong skewness of numeric variables: fitting encoders and scalers on the entire dataset would leak information from the test set into training transformations.

A key numeric outcome of this step is the dimensionality expansion introduced by encoding. In the **supervised branch**, the preprocessed training matrix becomes **(175,341 × 194)** and the preprocessed test matrix becomes **(82,332 × 194)**. This reflects the original 42 features being expanded through one-hot encoding into a higher-dimensional sparse representation. In the **normal-only anomaly branch**, the preprocessed normal training matrix becomes **(56,000 × 65)** and the preprocessed test matrix becomes **(82,332 × 65)**. The lower dimensionality here arises from the fact that one-hot encoding is fitted on normal-only data; categories absent from normal training traffic do not appear as encoded columns in this branch. This design choice is consistent with anomaly modelling assumptions (learning only from benign behaviour), but it also implies that the supervised and anomaly branches operate in different feature spaces (194 vs 65 dimensions), which must be kept in mind when comparing models across paradigms: performance differences may reflect both algorithm choice and representation differences induced by the training regime.

Overall, the pre-processing stage confirms that the dataset is clean (no duplicates, no missing values), quantifies substantial class imbalance and a long-tailed attack taxonomy, establishes a principled split between supervised and normal-only learning regimes, and constructs leakage-safe preprocessing pipelines that produce well-defined numeric feature matrices for subsequent modelling. These outputs collectively justify later methodological choices in the Modelling, Evaluation, and Tuning chapters, where the consequences of imbalance, high-dimensional encoding, and heavy-tailed numeric distributions become central to performance trade-offs.

3. Modelling

The modelling stage operationalises the intrusion detection task under two complementary paradigms—supervised classification and anomaly detection—implemented through two aligned data-processing branches. The supervised branch learns a direct mapping from preprocessed network-flow features to the binary target **label** (0 = normal, 1 = attack), while the anomaly branch learns a baseline representation of benign behaviour from normal-only traffic and flags deviations as suspicious using anomaly scores or reconstruction errors. This dual design is well suited to network security settings because it supports both label-driven discrimination and label-sparse detection, which is particularly relevant when attack patterns evolve, when previously unseen attack types appear, or when labelled examples are incomplete. Throughout this chapter, the emphasis is placed on model specification and numerical setup—such as sample sizes, feature dimensionality, scoring functions, decision rules, and thresholding policies—while performance outcomes are

deferred to the Evaluation chapter even when the same notebook sections generate diagnostic artefacts.

3.1. Supervised Branch — Baseline Models (No Resampling)

All supervised baselines were trained on the full labelled training split using the supervised preprocessing representation derived earlier. After one-hot encoding the categorical attributes (`proto`, `service`, `state`) and scaling numerical variables, the supervised design matrices contain **175,341 training samples** and **82,332 test samples**, with **194 input features**. A consistent prediction interface was adopted across supervised models to ensure comparability and to support later threshold-based analysis: for probabilistic classifiers, hard predictions are obtained using the standard rule $\hat{y} = 1$ if $P(\text{attack}) \geq 0.5$, otherwise $\hat{y} = 0$, while continuous probability scores are retained for later ROC and Precision–Recall construction. For models that do not inherently output calibrated probabilities, notably LinearSVC, probability-like scores are obtained through calibration so that the model can be analysed under the same threshold-sweep framework as probabilistic baselines. This modelling choice is operationally relevant for IDS deployment, where decision thresholds are often tuned to balance missed attacks against alert volume.

3.1.2. Logistic Regression (baseline)

Logistic Regression was used as an interpretable linear probabilistic baseline configured with `max_iter = 2000` and `class_weight = "balanced"`, which explicitly compensates for class imbalance by reweighting the loss contributions of normal and attack samples during optimisation. Because it outputs calibrated class probabilities, it integrates directly with the unified 0.5 decision threshold for hard labels and produces continuous scores for later threshold-sweep diagnostics. In modelling terms, it provides a stringent test of how much separability is achievable with a single linear decision boundary in the **194-dimensional** transformed feature space, where categorical network descriptors have been expanded into sparse indicator variables and numerical attributes have been standardised.

3.1.2. Random Forest (baseline)

Random Forest was included as a high-capacity non-linear baseline configured with `n_estimators = 300`, `n_jobs = -1` for parallelism, and a fixed `random_state = 42` for reproducibility. The model constructs 300 decision trees trained on bootstrap samples with feature subsampling, which reduces variance relative to a single tree and enables robust modelling of non-linear interactions among network-flow attributes. Numerically, increasing the number of trees stabilises probability estimates via averaged voting; therefore, the ensemble size (300) is a meaningful modelling parameter that controls the bias–variance profile and the smoothness of the predicted probability surface in high-dimensional spaces.

3.1.3. Gaussian Naive Bayes

Gaussian Naive Bayes was incorporated as a minimal-complexity generative classifier with no major hyperparameter tuning, providing a benchmark under strong simplifying assumptions—namely conditional feature independence and Gaussian likelihoods. Even though pre-processing indicates correlated numeric blocks (e.g., packet and byte counters) and strongly skewed distributions, this model remains useful for quantifying whether coarse distributional differences between normal and attack traffic are sufficient to yield discriminative signal when projected into the standardised feature space. Its modelling role is therefore comparative: it establishes a low-capacity reference point against which more expressive models can be contextualised.

3.1.4. Decision Tree

A single Decision Tree was trained with `class_weight = "balanced"` and `max_depth = None`, allowing the tree to expand until stopping criteria are met. In modelling terms, this configuration permits the tree to discover rule-like partitions that combine categorical indicators (from one-hot encoding) with numerical thresholds (from standardised continuous features). The use of balanced class weights is numerically important because it increases the penalty for misclassifying minority-class observations during split optimisation, which can shift the learned partitioning toward better coverage of normal traffic despite attack dominance in the training set.

3.1.5. ExtraTrees (Extremely Randomised Trees)

ExtraTrees was used as a randomised tree ensemble configured with `n_estimators = 500` and `n_jobs = -1`. Relative to Random Forest, ExtraTrees injects additional randomness at split selection, which can reduce variance and improve generalisation when many predictors are correlated or when multiple near-equivalent split points exist. The ensemble size (500 trees) is a meaningful numeric design choice: it increases stability of ensemble-averaged probabilities and reduces the sensitivity of the model to the stochasticity introduced by randomised splits.

3.1.6. Gradient Boosting (sklearn)

A Gradient Boosting classifier was included using sklearn defaults with `random_state = 42`. Boosting differs fundamentally from bagging-based ensembles because it builds trees sequentially, each correcting the residual structure not captured by prior learners. Although the baseline configuration does not expose the full tuning space (e.g., learning rate, number of estimators, and tree depth), the inclusion of boosting in the modelling suite is numerically relevant because its additive nature can model finer decision boundaries in the 194-dimensional representation and often benefits strongly from subsequent hyperparameter optimisation.

3.1.7. Linear SVM (LinearSVC) with Probability Calibration

A linear SVM was implemented via LinearSVC with `class_weight = "balanced"` and `random_state = 42`, and it was wrapped in a calibration layer using

`CalibratedClassifierCV(method="sigmoid", cv=3)`. Numerically, the calibration configuration indicates that probability mapping is learned using **3-fold cross-validation**, producing a probability-like output that is compatible with the unified decision threshold $P(\text{attack}) \geq 0.5$ and later threshold-sweep diagnostics. From a modelling standpoint, this SVM configuration is well suited to the high-dimensional, partially sparse representation induced by one-hot encoding, where large-margin separation can be effective even when feature interactions are complex.

3.2. Supervised Branch — SMOTE Imbalance-Aware Training

Imbalance-aware modelling was introduced via SMOTE because the labelled training split is substantially skewed: **119,341 attack** samples versus **56,000 normal** samples, making normal traffic the minority class. In this notebook configuration, SMOTE is instantiated with `random_state = 42` and default settings (including the default sampling strategy that oversamples the minority class toward parity with the majority class). A critical modelling constraint is leakage safety: SMOTE must be applied **only on training data** and only after a consistent numeric feature representation exists. The workflow therefore uses an imbalanced-learn pipeline where preprocessing is fitted on the training split, the training data is transformed into the supervised **194-dimensional** space, SMOTE generates synthetic minority (normal) samples in that transformed space, and the classifier is then fitted on the resampled training matrix. The test set remains untouched and is only transformed via the preprocessing step learned on training data, ensuring that the test distribution is preserved for evaluation.

3.2.1. Random Forest + SMOTE

Random Forest under SMOTE was configured with `n_estimators = 500`, `n_jobs = -1`, and `random_state = 42`, representing a deliberate increase in ensemble size relative to the baseline Random Forest (300 trees). Numerically, increasing the number of trees in the SMOTE setting improves stability when training data has been synthetically expanded, because ensemble averaging reduces sensitivity to local artefacts that may arise from interpolated samples. Conceptually, this model tests whether balancing the training distribution in a high-capacity non-linear ensemble produces a more symmetric decision surface for normal and attack regions in the transformed feature space.

3.2.2. Logistic Regression + SMOTE

Logistic Regression under SMOTE was trained with `max_iter = 2000` and `random_state = 42`. Unlike the baseline Logistic Regression, class weighting is not the primary imbalance mechanism here because resampling is used to increase minority representation directly. Numerically, the combination of a fixed maximum iteration count and SMOTE-based balancing aims to produce a stable linear separator whose placement is driven by a more balanced effective training distribution rather than by majority-class dominance.

3.3. Anomaly Branch — Classical Unsupervised Anomaly Detectors (Normal-Only Training)

To complement supervised learning with approaches capable of detecting novel or drifting attacks, multiple unsupervised anomaly detection models were implemented using the normal-only branch. In this regime, detectors are fitted on **56,000 normal** training samples and applied to the mixed test set of **82,332** samples. The anomaly branch uses a preprocessing pipeline fitted strictly on normal traffic, meaning the encoded feature space reflects categories observed in benign behaviour; this is consistent with anomaly modelling assumptions and helps ensure that score distributions and thresholds are derived exclusively from normal baselines.

3.3.1. Shared Modelling Principle: Score Standardisation and p95 Thresholding

A unifying modelling principle across anomaly detectors is the conversion of a continuous anomaly score into a binary decision via a consistent thresholding policy. Each method produces a score such that larger values correspond to greater abnormality (in the notebook, scores are explicitly inverted when necessary, e.g., by negating `score_samples()` or `decision_function()` outputs). The operational decision boundary is then defined using the **95th percentile (p95)** of the normal-training score distribution, yielding the rule: flag as attack if `anomaly_score ≥ p95(normal_train_scores)`. Numerically, this thresholding policy implies an expected upper-tail acceptance of approximately **5%** on normal training data under score stationarity, providing a transparent control knob over detector strictness and enabling comparability across heterogeneous scoring mechanisms.

3.3.2. Isolation Forest

Isolation Forest was configured with `n_estimators = 600`, `contamination = "auto"`, `random_state = 42`, and `n_jobs = -1`. The notebook defines the anomaly score as the negated output of `score_samples()`, ensuring a consistent “higher = more anomalous” convention. The p95 threshold computed on normal training scores is **0.5102265453**, which becomes the explicit operational cut-off. Numerically, the large number of estimators (600) increases stability of average isolation behaviour and reduces variance in anomaly scoring across runs.

3.3.3. One-Class SVM

One-Class SVM was configured with an RBF kernel and explicit regularisation and scale choices: `kernel = "rbf"`, `nu = 0.05`, and `gamma = "scale"`. In this formulation, `nu = 0.05` places an upper bound on the fraction of training errors and acts as a capacity/strictness parameter for the learned normal boundary. Scores are obtained as the negated decision function so that higher values indicate abnormality. The p95 threshold on normal training scores is **0.0002631129**, providing a numeric boundary for converting continuous boundary-violation scores into discrete alerts.

3.3.4. Local Outlier Factor — LOF

LOF was used in novelty detection mode with `n_neighbors = 35` and `novelty = True`, ensuring that the model is fitted on normal-only data and used to score unseen samples. The anomaly score is derived by negating the LOF decision function so that larger values reflect greater local density deviation relative to the normal baseline. The p95 threshold obtained from normal training scores is **-0.1278463744**, which becomes the operational cut-off. Numerically, the neighbourhood size (35) controls the scale of locality: larger values smooth local density estimates, while smaller values make the detector more sensitive to micro-structure.

3.3.5. kNN Distance-Based Anomaly Score

A kNN anomaly score was implemented using `k = 10` nearest neighbours via `NearestNeighbors`, with the score defined as the distance to the **10th nearest** normal neighbour. This produces a geometric notion of abnormality: points far from normal clusters have larger kNN distances. The p95 threshold computed on normal training distances is **1.8633563702**, providing a numeric boundary for flagging points that lie in the extreme tail of distance from the normal reference set. Numerically, the choice of `k = 10` balances local sensitivity with robustness to noise: it is local enough to capture cluster structure but large enough to avoid reacting to single-neighbour instabilities.

3.3.6. PCA Reconstruction Error

PCA-based anomaly detection was configured with `n_components = 0.95`, meaning the PCA model retains enough principal components to explain **95% of the variance** in normal training data. Reconstruction is performed by projecting into the retained subspace and inverting back into the original transformed space, with anomaly score defined as the mean squared reconstruction error per sample. The p95 threshold computed on normal training reconstruction errors is **0.0833766210**, which becomes the operational cut-off. Numerically, the “95% variance retained” criterion ties model complexity directly to explained variance rather than a fixed number of components, providing an adaptive linear baseline that is conceptually aligned with reconstruction-based detection.

3.3.7. EllipticEnvelope / Robust Covariance

EllipticEnvelope was included as a classical robust covariance baseline configured with `contamination = 0.05` and `random_state = 42`. Because its decision function is larger for more normal points, the notebook inverts it to produce an anomaly score where larger values indicate greater abnormality. The p95 threshold computed on normal training scores is **-9.257e-07**, which becomes the explicit operational boundary. Numerically, setting contamination to 0.05 aligns the model’s internal outlier expectation with the external p95 thresholding principle, creating a coherent “5% tail” modelling assumption across detectors.

3.4. Anomaly Branch — Deep Learning Autoencoder (Reconstruction-Based Detection)

A deep learning autoencoder was implemented as a non-linear reconstruction-based anomaly detector trained on normal-only traffic to learn a latent manifold of benign behaviour. The notebook performs a stratified validation split from the training data using `test_size = 0.2`, which yields approximately **35,069 validation samples** and **140,272 training samples** (given a total of 175,341), preserving label proportions through stratification. The autoencoder itself is trained only on the **normal subset** of the training partition, which is approximately **80% of 56,000 normal samples** (i.e., about **44,800 normal flows**), consistent with the observed training loop size of **88 batches per epoch** under the chosen batch size. Preprocessing for the autoencoder is fitted strictly on the normal training subset and produces a numeric input representation with an explicitly printed dimensionality of **63 features**, ensuring that the network receives a consistent scaled and encoded input space aligned with the anomaly branch.

3.4.1. Architecture and Optimisation Configuration

The autoencoder architecture is defined symmetrically with a bottleneck and dropout regularisation: **63** → **128** → **64** → **32** → **64** → **128** → **63**, where the bottleneck width is explicitly set to **32** and dropout is set to **0.1** after the first 128-unit layer. The model is trained using the Adam optimiser with a fixed **learning rate of 1e-3** and mean squared error as the reconstruction loss. Training is configured for **25 epochs** with a **batch size of 512**, and validation loss is monitored using a normal-only validation subset to ensure that the reconstruction objective remains anchored in benign behaviour. An EarlyStopping callback is included with **patience = 3** and `restore_best_weights = True`, meaning that if validation loss does not improve for three consecutive epochs, the training procedure will revert to the best-performing weights on the monitored normal-only validation criterion.

3.4.2. Training Dynamics (Numerical Convergence Profile)

The recorded training trace provides a clear numeric convergence pattern. At **epoch 1**, training loss begins at **0.4945** with validation loss **0.1086**, indicating substantial initial reconstruction error before the network learns a stable mapping. By **epoch 2**, training loss drops sharply to **0.0990** and validation loss to **0.0503**, reflecting rapid early learning. This improvement continues more gradually across epochs, with training loss reaching **0.0321** and validation loss **0.0205** by **epoch 5**, and then stabilising further toward the end of training. By **epoch 25**, the loss values converge to **0.0105 (train)** and **0.0059 (validation)**, indicating a consistent reduction in reconstruction error on both training and normal-only validation data and supporting the modelling choice of a moderate bottleneck and dropout regularisation under the selected learning rate and batch size.

3.4.3. Threshold Selection Strategy (Validation-Calibrated Operational Cut-off)

To operationalise continuous reconstruction errors into binary intrusion alerts, the notebook defines reconstruction error as the mean squared difference between the input and reconstructed vectors and then selects a threshold using a validation-calibrated search over candidate cut-offs. Candidate thresholds are drawn from the distribution of reconstruction errors on **normal-only training errors** at percentiles [90, 93, 95, 97, 99, 99.5], which explicitly explores increasingly strict boundaries in the upper tail of normal reconstruction behaviour. The final threshold is selected on a **mixed validation set** (containing both normal and attack samples) by choosing the candidate that maximises an objective on validation; the numeric threshold ultimately used in the notebook is the tuned cut-off previously reported in your workflow (**0.01007392900064587**) as the operational boundary for flagging attacks based on reconstruction error. While the Evaluation chapter will quantify the resulting outcomes, the modelling contribution here is the explicit specification of how reconstruction errors are converted into decisions and how the cut-off is constrained to a small set of percentile-based candidates derived strictly from normal training behaviour.

3.5. Modelling Summary

Across sections 9, 10, 12, and 14, the modelling workflow establishes a broad and methodologically coherent intrusion detection framework spanning supervised classifiers, imbalance-aware supervised learning via SMOTE, classical anomaly detection under normal-only training with p95 operational thresholding, and deep reconstruction-based detection via an autoencoder with explicitly defined architecture, optimisation settings, convergence dynamics, and validation-calibrated threshold selection. The chapter records numeric specifications at every stage—including training/test sample sizes, supervised feature dimensionality (**194**), anomaly-model hyperparameters (e.g., Isolation Forest **600** trees, LOF neighbourhood **35**, kNN **k = 10**, PCA **95% variance**), method-specific p95 thresholds, and autoencoder input dimensionality (**63**), batch size (**512**), epoch count (**25**), learning rate (**1e-3**), dropout (**0.1**), and final loss values (**0.0105/0.0059**). These quantified modelling choices define the operational behaviour of each detector and prepare the ground for the Evaluation chapter, where these architectures and thresholds are assessed comparatively using standard IDS performance artefacts and trade-off analysis.

4. Evaluation

The evaluation stage assesses all models on the held-out **UNSW-NB15 test split (n = 82,332)** using the binary target **label1** (0 = normal, 1 = attack). The test set is composed of **37,000 normal flows** and **45,332 attack flows**, which means the positive class (“attack”) is slightly more frequent in the test distribution. Model quality is therefore analysed not only through overall **accuracy**, but also through class-conditional behaviour, particularly **attack precision, attack recall (detection rate), and attack F1-score**, because intrusion detection systems must minimise missed attacks (false negatives) while keeping false alerts (false positives) operationally manageable. Confusion matrices are interpreted in terms of **TN/FP for normal traffic** and **TP/FN for attacks**, and the false positive rate (**FPR = FP / (FP + TN)**) and false negative rate (**FNR = FN / (FN + TP)**) are used to characterise the alerting trade-off. The numeric counts below follow directly from the model outputs and the confusion

matrices produced in the notebook (minor ± 1 differences may occur due to metric rounding in printed reports).

4.1. Supervised Branch — Baseline Models (No Resampling)

4.1.1. Logistic Regression (baseline)

Logistic Regression achieves an **accuracy of 0.8358**, with **attack precision = 0.8028**, **attack recall = 0.9304**, and **attack F1 = 0.8619**. This indicates that the model detects the majority of attack flows (high recall), but produces a non-trivial number of false positives among normal traffic. Numerically, the confusion matrix corresponds to approximately **TP $\approx$ 42,177** and **FN $\approx$ 3,155** for attacks, while normal traffic yields **TN $\approx$ 26,636** and **FP $\approx$ 10,364**. This translates to **FNR $\approx$ 6.96%** (missed attacks) and **FPR $\approx$ 28.01%** (normal flows incorrectly flagged). Operationally, this is a recall-oriented baseline: it prioritises detecting attacks but at the cost of a relatively high alert rate on benign traffic.

4.1.2. Random Forest (baseline)

Random Forest improves overall performance with **accuracy = 0.8696**, **attack precision = 0.8155**, **attack recall = 0.9865**, and **attack F1 = 0.8929**. The most notable gain is the extremely high attack recall, meaning very few attacks are missed. Quantitatively, the model produces approximately **TP $\approx$ 44,720** and **FN $\approx$ 612**, corresponding to **FNR $\approx$ 1.35%**, which is excellent from a detection perspective. However, normal discrimination remains moderate: **TN $\approx$ 26,880** and **FP $\approx$ 10,120**, yielding **FPR $\approx$ 27.35%**. This profile is characteristic of strong IDS classifiers trained on labelled data: very high detection rate, but still a considerable false-alarm burden.

4.1.3. Gaussian Naive Bayes

Gaussian Naive Bayes performs poorly as an IDS detector under the binary setting, with **accuracy = 0.5562**, **attack precision = 0.9998**, **attack recall = 0.1941**, and **attack F1 = 0.3251**. The model is extremely conservative in predicting “attack”: when it predicts attack it is almost always correct (near-perfect precision), but it fails to detect the vast majority of attacks. Numerically, this corresponds to approximately **TP $\approx$ 8,799** and **FN $\approx$ 36,533**, implying **FNR $\approx$ 80.59%**. In contrast, it classifies normal traffic almost perfectly (**TN $\approx$ 36,996**, **FP $\approx$ 4**, **FPR $\approx$ 0.01%**). This behaviour is not operationally acceptable for intrusion detection because the cost of missed attacks is typically far higher than the cost of additional false positives, and here most attacks would go undetected.

4.1.4. Decision Tree

The single Decision Tree achieves **accuracy = 0.8619**, with **attack precision = 0.8180**, **attack recall = 0.9636**, and **attack F1 = 0.8849**. The model detects attacks strongly but not at the same level as Random Forest. Numerically, the confusion matrix corresponds to approximately **TP $\approx$ 43,682** and **FN $\approx$ 1,650** (**FNR $\approx$ 3.64%**), while normal classification yields

TN $\approx$ 27,284 and **FP $\approx$ 9,716 (FPR $\approx$ 26.26%)**. Compared to Logistic Regression, the tree improves attack recall and slightly reduces missed attacks, but still produces a sizable number of false alarms on normal traffic.

4.1.5. ExtraTrees (Extremely Randomised Trees)

ExtraTrees yields **accuracy = 0.8626**, **attack precision = 0.8085**, **attack recall = 0.9833**, and **attack F1 = 0.8874**. Its strength is again high attack recall (few missed attacks), with approximately **TP $\approx$ 44,575** and **FN $\approx$ 757 (FNR $\approx$ 1.67%)**. However, it tends to generate more false positives than Decision Tree, with **TN $\approx$ 26,444** and **FP $\approx$ 10,556 (FPR $\approx$ 28.53%)**. This suggests that the additional split randomisation increases sensitivity to attack patterns but can also enlarge the region classified as attack, increasing alert volume.

4.1.6. Gradient Boosting

Gradient Boosting reaches **accuracy = 0.8550**, with **attack precision = 0.7984**, **attack recall = 0.9855**, and **attack F1 = 0.8822**. It maintains a very high detection rate, with approximately **TP $\approx$ 44,675** and **FN $\approx$ 657 (FNR $\approx$ 1.45%)**, but exhibits the highest false positive propensity among the stronger supervised baselines here, with **TN $\approx$ 25,722** and **FP $\approx$ 11,278 (FPR $\approx$ 30.48%)**. In an operational IDS, this would be a high-sensitivity model that may require threshold adjustment or additional post-filtering to control alert burden.

4.1.7. Linear SVM (calibrated)

The calibrated Linear SVM shows **accuracy = 0.8085**, with **attack precision = 0.7455**, **attack recall = 0.9904**, and **attack F1 = 0.8507**. This model achieves the **highest attack recall** among the supervised baselines, implying a very low missed-attack rate: approximately **TP $\approx$ 44,897** and **FN $\approx$ 435 (FNR $\approx$ 0.96%)**. However, the cost is a substantial increase in false positives: **TN $\approx$ 21,671** and **FP $\approx$ 15,329**, producing **FPR $\approx$ 41.43%**. This indicates that the decision boundary is highly permissive toward predicting “attack,” which can be unacceptable operationally due to alert overload even though detection is extremely strong.

4.2. Supervised Branch — SMOTE Imbalance-Aware Training

4.2.1. Random Forest + SMOTE

Random Forest trained with SMOTE achieves the strongest overall supervised result in this set, with **accuracy = 0.8840**, **attack precision = 0.8367**, **attack recall = 0.9808**, and **attack F1 = 0.9030**. Numerically, this corresponds to approximately **TP $\approx$ 44,462** and **FN $\approx$ 870 (FNR $\approx$ 1.92%)**, while normal traffic yields **TN $\approx$ 28,320** and **FP $\approx$ 8,680 (FPR $\approx$ 23.46%)**. Compared with the baseline Random Forest, this configuration slightly increases missed attacks (FN rises from $\sim$ 612 to $\sim$ 870), but it produces a clear improvement in normal discrimination (FP decreases from $\sim$ 10,120 to $\sim$ 8,680). The net effect is a better balance

between detection capability and false-alarm control, which is reflected both in the improved accuracy and in the strong attack F1-score.

4.2.2. Logistic Regression + SMOTE

Logistic Regression + SMOTE produces **accuracy = 0.8379**, with **attack precision = 0.8048**, **attack recall = 0.9315**, and **attack F1 = 0.8637** (from the classification report). Numerically, **TP $\approx$ 42,227** and **FN $\approx$ 3,105** (**FNR $\approx$ 6.85%**), while **TN $\approx$ 26,758** and **FP $\approx$ 10,242** (**FPR $\approx$ 27.68%**). This is extremely close to the baseline Logistic Regression, indicating that in this configuration SMOTE does not substantially shift the linear decision boundary's operational trade-off. In practical terms, resampling benefits are more pronounced for the high-capacity Random Forest than for the linear model in this experiment.

4.3. Anomaly Branch — Classical Unsupervised Anomaly Detectors (p95 Thresholding)

4.3.1. Isolation Forest (p95 threshold)

Isolation Forest yields **accuracy = 0.5936**, with **attack precision = 0.8585**, **attack recall = 0.3135**, and **attack F1 = 0.4593**. The detector has a strong bias toward classifying traffic as normal under the p95 thresholding rule. Quantitatively, it detects only about one-third of attacks (**TP $\approx$ 14,212**) while missing a very large number (**FN $\approx$ 31,120**, **FNR $\approx$ 68.65%**). In contrast, it classifies normal traffic well (**TN $\approx$ 34,658**, **FP $\approx$ 2,342**, **FPR $\approx$ 6.33%**). This model is therefore conservative and low-noise but fails to provide adequate attack coverage, suggesting that its learned “normality” structure overlaps heavily with many attack patterns under the selected thresholding scheme.

4.3.2. One-Class SVM (p95 threshold)

One-Class SVM behaves similarly to Isolation Forest with **accuracy = 0.5834**, **attack precision = 0.8170**, **attack recall = 0.3137**, and **attack F1 = 0.4533**. Numerically, it produces **TP $\approx$ 14,221** and **FN $\approx$ 31,111** (**FNR $\approx$ 68.63%**), while normal traffic yields **TN $\approx$ 33,814** and **FP $\approx$ 3,186** (**FPR $\approx$ 8.61%**). Compared to Isolation Forest, it produces more false positives and does not improve attack coverage, making it similarly unsuitable as a standalone IDS detector under this thresholding regime.

4.3.3. LOF novelty (p95 threshold)

LOF novelty provides the strongest classical anomaly performance in this suite, with **accuracy = 0.8163**, **attack precision = 0.8873**, **attack recall = 0.7633**, and **attack F1 = 0.8207**. The confusion matrix indicates **TP $\approx$ 34,602** and **FN $\approx$ 10,730** (**FNR $\approx$ 23.67%**), while normal traffic yields **TN $\approx$ 32,604** and **FP $\approx$ 4,396** (**FPR $\approx$ 11.88%**). This model achieves a substantially better balance between detecting attacks and controlling false positives than the other anomaly detectors, consistent with the idea that local-density deviation captures meaningful abnormality structure in network traffic when trained on normal-only data.

4.3.4. kNN anomaly (k = 10, p95 threshold)

The kNN distance-based detector achieves **accuracy = 0.7559**, with **attack precision = 0.8775**, **attack recall = 0.6470**, and **attack F1 = 0.7448**. Numerically, **TP $\approx$ 29,330** and **FN $\approx$ 16,002** (**FNR $\approx$ 35.30%**), while **TN $\approx$ 32,908** and **FP $\approx$ 4,092** (**FPR $\approx$ 11.06%**). Compared with LOF, kNN produces a similar false positive rate but detects substantially fewer attacks, suggesting that distance to nearest normal neighbours captures normality reasonably well but is less effective than density-based scoring for separating many attack behaviours.

4.3.5. PCA reconstruction error (p95 threshold)

PCA reconstruction error performs weakly with **accuracy = 0.5406**, **attack precision = 0.7524**, **attack recall = 0.2468**, and **attack F1 = 0.3717**. Numerically, **TP $\approx$ 11,188** and **FN $\approx$ 34,144** (**FNR $\approx$ 75.32%**), while **TN $\approx$ 33,318** and **FP $\approx$ 3,682** (**FPR $\approx$ 9.95%**). This indicates that a linear reconstruction model explaining benign variance is insufficient to meaningfully separate a large portion of attack traffic from normal flows, likely because many attack patterns still lie close to the linear subspace learned from normal behaviour.

4.3.6. EllipticEnvelope (p95 threshold)

EllipticEnvelope yields **accuracy = 0.7388**, with **attack precision = 0.8870**, **attack recall = 0.6023**, and **attack F1 = 0.7174**. Numerically, **TP $\approx$ 27,303** and **FN $\approx$ 18,029** (**FNR $\approx$ 39.77%**), while **TN $\approx$ 33,522** and **FP $\approx$ 3,478** (**FPR $\approx$ 9.40%**). This model maintains a relatively low false positive rate and good precision but sacrifices attack coverage. It therefore provides a conservative anomaly detection baseline with moderate IDS utility when low alert volume is prioritised.

4.4. Anomaly Branch — Autoencoder (Threshold Tuned on Validation)

The deep autoencoder (trained on normal-only data and thresholded using a validation-tuned cut-off) achieves **accuracy = 0.8413**, with **attack precision = 0.8638**, **attack recall = 0.8451**, and **attack F1 = 0.8543**. Numerically, this corresponds to approximately **TP $\approx$ 38,310** and **FN $\approx$ 7,022** (**FNR $\approx$ 15.49%**), while normal flows yield **TN $\approx$ 30,962** and **FP $\approx$ 6,038** (**FPR $\approx$ 16.32%**). Compared to the strongest classical anomaly detector (LOF), the autoencoder increases attack recall markedly (from 0.7633 to 0.8451), but it also increases false positives (FPR rises from $\sim$ 11.88% to $\sim$ 16.32%). Relative to supervised Random Forest + SMOTE, the autoencoder yields substantially more missed attacks (FN $\sim$ 7,022 vs $\sim$ 870), but it produces fewer false positives (FP $\sim$ 6,038 vs $\sim$ 8,680), reflecting the expected trade-off between normal-only modelling and fully supervised discrimination learning. The reconstruction-error based figures produced in the notebook support this numeric interpretation by showing that attacks tend to concentrate at higher reconstruction errors than normal traffic, but overlap remains, which necessitates a threshold that inevitably trades detection for false alarms.

4.5. Cross-Model Evaluation Summary

Across supervised models, the strongest operational balance in this experiment is achieved by **Random Forest + SMOTE**, which attains the highest accuracy (**0.8840**) while maintaining high attack recall (**0.9808**) and simultaneously reducing false positives relative to the baseline Random Forest (**FPR ~23.46% vs ~27.35%**). The baseline Random Forest remains the most aggressive detector with extremely low missed-attack rate (**FNR ~1.35%**) but at a higher false-alarm cost. Linear SVM (calibrated) achieves the highest attack recall (**0.9904**) and lowest FNR (**~0.96%**) but suffers from an operationally heavy false-positive rate (**~41.43%**), which would likely require post-processing or threshold tightening in a real IDS. Gaussian Naive Bayes is demonstrably unsuitable due to its extremely high missed-attack rate despite near-zero false positives.

Across anomaly detectors, **LOF novelty** provides the best classical unsupervised balance (accuracy **0.8163**, attack recall **0.7633**, FPR **~11.88%**), while Isolation Forest and One-Class SVM are highly conservative under p95 thresholding, producing low false positive rates but missing the majority of attacks. The **autoencoder** is the strongest normal-only model in terms of attack coverage (attack recall **0.8451**) and achieves an accuracy (**0.8413**) comparable to linear supervised baselines, but it still cannot match the detection rate of the best supervised ensembles, which benefit from explicit labelled attack structure. These results collectively highlight the expected IDS pattern: supervised ensembles excel when labelled attacks are available, while anomaly detection provides an alternative that can reduce dependence on attack labels but typically trades off detection completeness for lower false-alarm rates and broader generalisation assumptions.

5. Model Tuning

Model tuning was conducted to refine the strongest candidate architectures from both the supervised and anomaly-detection branches, using procedures that are explicitly designed to avoid train–test leakage and to ensure that optimisation decisions are based solely on development data. The tuning workflow corresponds to notebook sections **11)** and **13)** and follows two complementary strategies. In the supervised setting, hyperparameters are selected via stratified cross-validation using an F1-oriented objective, reflecting the need to balance missed attacks (false negatives) and false alerts (false positives) under class imbalance. In the anomaly-detection setting, tuning is performed by calibrating both detector configuration and its decision threshold on a mixed validation split, with thresholds derived from the score distribution of normal training traffic and selected to maximise validation F1. In both cases, the final tuned configurations are then carried forward for test-set evaluation and model selection, but the primary focus of this chapter is the tuning methodology and its numeric outcomes on development data.

5.1. Supervised Tuning — Random Forest + SMOTE (Randomised Search with Stratified CV)

For supervised model tuning, the notebook uses a pipeline that integrates preprocessing, resampling, and classification into a single leakage-safe training object. The tuned pipeline is defined as **Preprocess** → **SMOTE** → **RandomForestClassifier**, where preprocessing applies the same supervised transformations established earlier (one-hot encoding for **proto**, **service**, **state** and scaling for numerical variables), SMOTE oversamples the

minority class within training only, and the Random Forest learns the final discriminative decision function. This pipeline-level design is numerically important because it ensures that synthetic samples generated by SMOTE are created only within the training folds of cross-validation, rather than being influenced by information from validation folds or the test set.

Hyperparameter optimisation is performed using **3-fold StratifiedKFold cross-validation**, configured with shuffling and a fixed random seed to ensure reproducibility. The search evaluates **12 candidate configurations**, resulting in **36 fitted models** in total (3 folds × 12 candidates), which provides meaningful exploration of the parameter space while remaining computationally feasible. The optimisation objective is **F1-score**, a suitable choice for intrusion detection because it balances precision (alert quality) and recall (attack coverage) and avoids the misleading stability of accuracy under imbalance.

The tuned search space explicitly targets Random Forest capacity and regularisation mechanisms that are known to affect IDS behaviour. The parameter grid includes the number of trees (**n_estimators** ∈ {300, 500, 800}), maximum depth (**max_depth** ∈ {None, 15, 25, 35}), split regularisation (**min_samples_split** ∈ {2, 5, 10}), leaf regularisation (**min_samples_leaf** ∈ {1, 2, 4}), and feature subsampling (**max_features** ∈ {"sqrt", "log2", None}). These parameters jointly control both the expressiveness of the ensemble (depth and number of estimators) and its generalisation behaviour (minimum split/leaf sizes and feature sampling), which is particularly relevant in a high-dimensional representation created by one-hot encoding.

The tuning procedure reports a **best cross-validated F1-score of 0.9689545749579894**, indicating that, within the training distribution and cross-validation regime, the RF+SMOTE pipeline can achieve very strong balanced detection performance. The reported **best hyperparameter configuration** is a medium-depth, moderately regularised ensemble: **n_estimators = 300, max_depth = 25, min_samples_split = 5, min_samples_leaf = 1, and max_features = None**. Numerically, this selection implies that the model benefits from allowing each split to consider the full feature set (max_features=None) while constraining tree growth through a finite depth (25) and a split requirement that prevents overly small partitions (min_samples_split=5). Compared to deeper or unconstrained trees, this configuration typically reduces overfitting to idiosyncratic training patterns, while still preserving sufficient capacity to model non-linear interactions among protocol/service indicators and traffic-statistic features. The cross-validation outcome therefore provides a principled, data-driven justification for the final supervised configuration taken forward to test evaluation.

5.2. Unsupervised Tuning — Isolation Forest Configuration and Validation-Calibrated Threshold

For anomaly-detection tuning, the notebook applies a different optimisation principle, reflecting the fact that unsupervised detectors do not learn a direct mapping to the binary label. Instead, the model produces a continuous anomaly score that must be converted into a binary decision via a threshold. The tuning objective is therefore defined over both (i) the detector's hyperparameters and (ii) the threshold used to declare a sample anomalous. To achieve this in a leakage-safe manner, the notebook first constructs a **mixed validation**

split from the training set using a **stratified 80/20 split** ($\text{test_size} = 0.2$), ensuring that label proportions are preserved in the validation data. Crucially, the preprocessing step is fitted only on the **normal subset of the training partition** (i.e., X_{tr} where $y_{\text{tr}} = 0$), consistent with the anomaly-detection assumption that the “normality model” should be learned strictly from benign behaviour. The fitted preprocessing is then applied to both the normal-only training partition and the mixed validation partition.

Within this framework, Isolation Forest is tuned over a compact grid of configurations that directly control ensemble stability and the sampling behaviour of tree construction. The grid varies $\text{n_estimators} \in \{200, 400, 800\}$ and $\text{max_samples} \in \{\text{"auto"}, 0.7, 1.0\}$, producing **9 candidate configurations**. For each candidate model, the notebook computes anomaly scores on the normal-only training partition and on the mixed validation set, then evaluates a set of candidate thresholds and selects the threshold that maximises **validation F1**. This design explicitly recognises that the “best” detector configuration depends not only on score ranking quality, but also on where the operational boundary is set in score space.

The tuning results show a clear numeric pattern: configurations with $\text{max_samples} = 1.0$ yield the strongest validation performance, and increasing the number of estimators beyond 400 provides only marginal improvements. The top-performing configuration is $\text{n_estimators} = 400$ with $\text{max_samples} = 1.0$, which achieves a **validation F1 of 0.812854** with a selected threshold $\text{best_thr} = 0.402508$. The next-best configurations are extremely close in validation performance, including $\text{n_estimators} = 800$, $\text{max_samples} = 1.0$ with **val F1 = 0.812708** and $\text{best_thr} = 0.402479$, and $\text{n_estimators} = 800$, $\text{max_samples} = 0.7$ with **val F1 = 0.809320** and $\text{best_thr} = 0.408951$. This clustering indicates that, once max_samples is sufficiently large, the detector’s ranking behaviour stabilises and the optimal threshold falls within a narrow numeric band around **0.40–0.41**.

In contrast, configurations using $\text{max_samples} = \text{"auto"}$ perform substantially worse on the mixed validation objective, with validation F1 values around **0.6203–0.6206** and thresholds in the **0.47–0.48** range. The magnitude of this gap is meaningful: it implies that sampling fewer observations per tree (as implied by “auto”) reduces the detector’s ability to separate attacks from normal flows under the chosen score construction and thresholding strategy, whereas using larger sampling fractions (0.7 or 1.0) yields a substantially better trade-off between precision and recall on validation. Importantly, this tuning framework produces not only a chosen Isolation Forest configuration but also an explicit, validation-justified operational threshold, making the anomaly detector deployable as a deterministic decision rule rather than as an uncalibrated score generator.

5.3. Tuning Summary

Overall, the tuning stage produces two numerically grounded outcomes that feed directly into the final model comparison and selection. In the supervised branch, the tuned **RF+SMOTE** pipeline achieves a **best 3-fold CV F1 of 0.96895** under **12 candidate evaluations**, with a best configuration that balances ensemble capacity and regularisation through **300 trees**, **depth 25**, and moderate split constraints. In the anomaly branch, Isolation Forest tuning identifies $\text{n_estimators} = 400$ and $\text{max_samples} = 1.0$ as the strongest configuration on a mixed validation objective, together with a narrowly defined operational threshold around **0.4025** that maximises validation F1 (**0.812854**). These tuned specifications provide a

principled bridge from model construction to final selection, ensuring that the models compared in the subsequent section are not only evaluated, but also calibrated and optimised using transparent, reproducible, and leakage-safe procedures.

6. Model Selection

Model selection was performed by integrating the evidence produced in the Modelling, Evaluation, and Model Tuning stages and choosing the detector(s) that best satisfy the operational objectives of intrusion detection on the UNSW-NB15 test set. Because the problem is inherently cost-sensitive—false negatives correspond to missed attacks, while false positives correspond to benign traffic incorrectly flagged and therefore consume analyst time—model selection is based on a balanced assessment of (i) **attack detection capability** (high recall / low FNR), (ii) **alert quality and workload** (precision and FPR), and (iii) **overall robustness and stability**, including whether performance is obtained through a leakage-safe training regime and whether tuning delivers consistent improvements. In addition, since the report compares supervised and anomaly-based paradigms, selection is considered under two practical deployment scenarios: a fully supervised setting where labelled attacks are available for training and a normal-only setting where models must generalise without direct exposure to attack labels.

Within the supervised family, the strongest test-set performance and the most favourable trade-off between missed attacks and false alarms is achieved by **Random Forest trained with SMOTE**. On the test set, this model attains the highest overall accuracy (**0.8840**) among the evaluated candidates, together with a strong attack-class balance reflected by **attack precision = 0.8367**, **attack recall = 0.9808**, and **attack F1 = 0.9030**. Numerically, the confusion matrix indicates **TP $\approx$ 44,462** and **FN $\approx$ 870**, implying a very low missed-attack rate (**FNR $\approx$ 1.92%**), while normal traffic yields **TN $\approx$ 28,320** and **FP $\approx$ 8,680**, corresponding to **FPR $\approx$ 23.46%**. This outcome is particularly important in IDS practice: the model preserves near-complete attack coverage while reducing false alarms relative to the baseline Random Forest (which produced roughly **FP $\approx$ 10,120**, **FPR $\approx$ 27.35%**). The SMOTE variant therefore provides a concrete operational benefit—fewer false alerts—without materially sacrificing the high detection rate that motivates the use of ensemble tree models for intrusion detection.

The choice of Random Forest + SMOTE is further supported by the supervised tuning results, which show that this pipeline is not only strong on the held-out test set but also optimisable in a controlled manner. The Randomised Search procedure using 3-fold stratified cross-validation yields a best CV F1 of **0.96895**, and the selected hyperparameters (300 trees, depth 25, min_samples_split 5, min_samples_leaf 1, max_features None) indicate a model that balances capacity with regularisation. Importantly, the tuning was performed within a leakage-safe pipeline where SMOTE is applied inside each training fold, ensuring that the performance gains are not artefacts of improper resampling. From a selection standpoint, this provides confidence that the final supervised model is both empirically strong and methodologically sound.

Although some supervised alternatives achieve extremely high attack recall, they were not selected due to less favourable operational trade-offs. In particular, the calibrated Linear SVM achieves the highest observed attack recall (**0.9904**) and the lowest missed-attack rate

(**FNR $\approx$ 0.96%**), but this comes at the cost of an exceptionally high false positive rate (**FPR $\approx$ 41.43%**, with **FP $\approx$ 15,329**), which would likely overwhelm an IDS pipeline in real-world conditions. Similarly, the baseline Random Forest attains near-perfect attack recall (**0.9865**) but generates more false positives than its SMOTE-enhanced counterpart. Gaussian Naive Bayes is excluded decisively because it misses the majority of attacks (attack recall $\sim$ 0.194), demonstrating that conservative prediction can yield deceptively high precision but unacceptable detection coverage. These comparisons reinforce the selection criterion that the best model is not the one with the highest recall alone, but the one that achieves strong recall while keeping false alarms within a manageable range.

In the anomaly-detection family, a separate “best” model is selected for the normal-only setting, where detectors must be trained without labelled attacks. Under this constraint, the strongest unsupervised candidate is the **deep autoencoder** trained on normal-only traffic with a validation-tuned reconstruction threshold. On the test set, the autoencoder achieves **accuracy = 0.8413** with **attack precision = 0.8638**, **attack recall = 0.8451**, and **attack F1 = 0.8543**. Numerically, it produces **TP $\approx$ 38,310** and **FN $\approx$ 7,022** (FNR $\approx$ 15.49%), and it yields **FP $\approx$ 6,038** on normal traffic (FPR $\approx$ 16.32%). While this detection rate is below the supervised ensembles, the false positive rate is meaningfully lower than that of the best supervised classifier, and the precision is comparatively high for a normal-only detector. This selection is justified by the modelling assumptions: because the autoencoder has not been trained on attack examples, it cannot be expected to match supervised discrimination, but it provides a strong anomaly baseline that remains deployable when labels are unavailable or when the system must detect deviations from a learned benign manifold.

Among classical anomaly detectors, **LOF novelty** emerges as the strongest non-deep baseline, achieving **accuracy = 0.8163** and attack recall **0.7633** with FPR $\approx$ 11.88%. However, it is not selected as the primary normal-only model because it yields substantially more missed attacks than the autoencoder (attack recall 0.7633 vs 0.8451) under the chosen p95 thresholding policy. Isolation Forest and One-Class SVM are not selected because they are overly conservative under p95 thresholding, detecting only about 31% of attacks and therefore failing to provide adequate coverage. PCA reconstruction error is excluded due to very low attack recall, indicating that linear reconstruction is insufficient for capturing the complexity of normal behaviour relative to attack variation. The tuned Isolation Forest configuration improves validation behaviour but, in the final test comparisons used here, does not surpass the autoencoder’s normal-only balance between attack coverage and false-alarm control.

Based on these results, the final model selection depends on the deployment regime. For a fully supervised IDS where labelled attack data is available and stable, the selected model is **Random Forest + SMOTE**, because it achieves the best overall test performance and delivers a favourable operational trade-off of **very low missed-attack rate ($\sim$ 1.92%)** with a reduced false-alarm rate relative to the baseline ensemble. For a label-sparse or anomaly-centric IDS setting where only benign traffic is reliably available for training, the selected model is the **normal-only autoencoder**, because it provides the strongest anomaly detection balance observed in this study, combining comparatively high attack recall (**0.8451**) with a moderate false positive rate (**$\sim$ 16.32%**) and a clearly defined validation-calibrated decision threshold. This two-model selection outcome reflects a realistic security engineering perspective: supervised ensembles are preferred when high-quality

labels exist, while reconstruction-based anomaly detection provides an effective alternative when the system must generalise beyond known attack signatures.

7. Conclusion

This report implemented and assessed an end-to-end intrusion detection workflow on the UNSW-NB15 dataset, combining supervised classification with normal-only anomaly detection in a leakage-safe and reproducible pipeline. The dataset was used in its predefined train/test split, with **175,341** training records and **82,332** test records, and pre-processing confirmed strong data integrity (no duplicates and no missing values). Exploratory analysis highlighted the practical characteristics that shape IDS design: substantial class imbalance at the binary level (normal vs attack), a long-tailed distribution across attack families, high-cardinality categorical fields (notably **proto**), and heavily skewed numeric traffic features with correlated blocks among packet/byte counters and TCP timing variables. These findings motivated the use of structured preprocessing via ColumnTransformer pipelines, the separation of supervised and anomaly branches, and threshold-based operating rules for anomaly detectors.

In the supervised branch, a diverse set of models spanning linear, probabilistic, margin-based, and ensemble decision functions was trained on a unified **194-dimensional** encoded representation. The evaluation results demonstrated that ensemble tree methods provide the strongest attack coverage under this dataset, while simpler distributional baselines can be overly conservative and miss attacks. Imbalance-aware training via SMOTE proved most beneficial for the high-capacity ensemble, improving the operational balance by reducing false alarms while preserving very high attack recall. In particular, **Random Forest + SMOTE** achieved the best overall supervised trade-off on the test set with **accuracy = 0.8840**, **attack recall = 0.9808** (missed-attack rate $\approx 1.92\%$), and a reduced false positive rate relative to the baseline Random Forest. The supervised tuning procedure reinforced this outcome by identifying a stable, moderately regularised configuration with strong cross-validated F1 performance, supporting confidence in the selected supervised solution.

In the anomaly-detection branch, multiple classical detectors were trained on **normal-only** traffic and operationalised using a consistent percentile-based thresholding policy (p95), enabling transparent comparability across heterogeneous scoring mechanisms. Results showed that several classical detectors were either too conservative (missing most attacks) or insufficiently expressive for the complexity of network behaviours. The strongest classical anomaly baseline was LOF in novelty mode, indicating that local density deviation can capture meaningful abnormality structure in this dataset. However, the most effective normal-only approach overall was the deep **autoencoder**, which leveraged non-linear reconstruction to model benign behaviour and detect deviations via a validation-tuned reconstruction threshold. The autoencoder achieved **accuracy = 0.8413** with **attack recall = 0.8451** and a moderate false positive rate, illustrating that reconstruction-based anomaly detection can provide a viable alternative when attack labels are unavailable, even though it cannot fully match the near-complete detection rates of supervised ensembles trained directly on attack examples.

Model selection therefore depends on the deployment regime. When high-quality labelled attack data is available, a supervised ensemble is the preferred choice; in this study, **Random Forest + SMOTE** offers the best balance between high detection rate and manageable false alarms. When labels are limited or when the detector must generalise beyond known signatures, the **normal-only autoencoder** provides the strongest anomaly-based option, combining strong attack coverage with a clearly defined, validation-calibrated decision boundary. This two-track conclusion aligns with realistic IDS engineering practice, where supervised detectors are often deployed for high-confidence detection and anomaly detectors complement them by providing broader coverage for novel behaviours.

Despite these strengths, several limitations should be noted. First, anomaly detectors were calibrated using percentile and validation-based thresholds, and their performance is inherently sensitive to score distribution stability; in real networks, **concept drift** may shift normal behaviour over time and require periodic recalibration. Second, the anomaly branch was trained on a normal-only feature space fitted on benign traffic, which reflects the intended modelling assumption but also implies representational differences from the supervised branch; future work could explore consistent feature spaces or hybrid semi-supervised approaches. Third, the evaluation considered the dataset's binary label, while the underlying attack taxonomy is diverse and long-tailed; a multi-class or per-attack-family analysis could reveal additional model strengths and weaknesses, especially for rare categories such as Worms or Shellcode.

Future improvements may therefore include threshold calibration under explicit operational constraints (e.g., targeting a maximum allowable false positive rate), probability calibration and cost-sensitive decision thresholds for supervised models, feature selection or representation learning to reduce dimensionality and improve robustness, and drift-aware re-training schedules for deployed IDS systems. Additionally, combining supervised ensembles with anomaly detectors in an ensemble-of-ensembles architecture—where anomaly scores act as a second-stage filter or as a novelty trigger—could provide a practical path toward high detection coverage with controlled alert volume. Overall, the results confirm that well-tuned supervised ensembles remain highly effective on UNSW-NB15, while reconstruction-based anomaly detection offers a credible and deployable alternative in label-sparse intrusion detection scenarios.

8. Resources

8.1. Course Books

- James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J. (2023) *An Introduction to Statistical Learning with Applications in Python*. Available at: <https://www.statlearning.com> (Accessed: 11 December 2025).
- Géron, A. (2019) *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. 2nd edn. Sebastopol, CA: O'Reilly Media.
- Chio, C. and Freeman, D. (2018) *Machine Learning and Security: Protecting Systems with Data and Algorithms*. Sebastopol, CA: O'Reilly Media.

- **Bhattacharyya, D.K. and Kalita, J.K.** (2013) Network Anomaly Detection: A Machine Learning Perspective. 1st edn. Boca Raton, FL: Chapman and Hall/CRC.
- **Mehrotra, K.G., Mohan, C.K. and Huang, H.** (2017) Anomaly Detection Principles and Algorithms. Cham: Springer International Publishing.
- **Momcheva, G., Sabra, A. and Rmeiti, N.** (2022) Machine Learning in Cybersecurity: A Textbook. Varna: Varna Free University “Chernorizetz Hrabar”, University Publishing House.
- **Benavides-Prado, D., Erfani, S., Fournier-Viger, P., Boo, Y.L. and Koh, Y.S.** (eds.) (2024) Data Science and Machine Learning. Communications in Computer and Information Science, 1943. Singapore: Springer Nature Singapore Pte Ltd.
- **Legg, P. and Giacinto, G.** (eds.) (2025) Machine Learning and Data Analytics for Cyber Security. Basel: MDPI.

8.2. Research articles

- **Moustafa, N. and Slay, J.** (2015) ‘UNSW-NB15: a comprehensive data set for network intrusion detection systems’, *Proceedings of the Military Communications and Information Systems Conference (MilCIS)*. (Available via ResearchGate).
- **Zoghi, Z. and Serpen, G.** (2021) UNSW-NB15 Computer Security Dataset: Analysis through Visualization. arXiv:2101.05067. Available at: <https://arxiv.org/pdf/2101.05067> (Accessed: 11 December 2025).

8.3. Official dataset website (Harvard)

- **UNSW Canberra Cyber** (n.d.) UNSW-NB15 dataset. Available at: <https://research.unsw.edu.au/projects/unsw-nb15-dataset> (Accessed: 11 December 2025).